

Programação Imperativa – Exame, 13 de Junho de 2025

1. Um conjunto de inteiros pode ser representado como uma união de intervalos. Por exemplo, o conjunto {1,2,3,4,5,6,10,11} pode ser representado pela união dos intervalos [1..6] e [10..11]. Implemente a função `int existe (int x, Intervalo c[], int N)` que dado um conjunto `c` com `N` intervalos guardados num array, testa se o número `x` pertence a esse conjunto. Assuma que o conjunto está representado de forma normalizada, não existindo intervalos que se intersectam e estando os intervalos ordenados de forma crescente. O tipo `Intervalo` está declarado da seguinte forma.

```
typedef struct {  
    int inf,sup;  
} Intervalo;
```

2. Implemente a função `int acrescenta (Intervalo a, Intervalo c[], int N)`, que dado um conjunto `c` com `N` intervalos guardados num array de forma normalizada (ordenados de forma crescente e sem haver intervalos que se intersectam), acrescenta o intervalo `a` a esse conjunto, mantendo a representação normalizada. A função deve devolver o número de intervalos usados para representar o conjunto resultante.

3. Defina uma função `void invertePals (char *frase)` que inverte cada uma das palavras de uma frase (assuma que uma palavra é uma sequência de caracteres alfabéticos). Por exemplo, se `char f[] = "Ser ou nao ser, eis a questao."`, após a invocação `invertePals(f)` o array `f` deve conter a string "reS uo oan res, sie a oatseuq."

4. Defina a função `int listToArray (LInt l, int v[], int N)` que dada uma lista `l` preenche o array `v` com os elementos da lista (sem alterar a lista). A função deverá preencher no máximo `N` elementos e retornar o número de elementos preenchidos. O tipo `LInt` está declarado da seguinte forma.

```
typedef struct no {  
    int valor;  
    struct no *prox;  
} *LInt;
```

5. Defina a função `int removeMenores(LInt *l)` que dada uma lista `l` remove dessa lista todos os nós que tenham um valor estritamente menor que o nó anterior na lista original. A função deve devolver o comprimento da lista resultante.

6. Defina uma função **ABin** `folhaMaisProfunda (ABin a)` que, dada uma árvore binária não vazia, calcula o endereço de uma das folhas que está no nível mais profundo. Na seguinte árvore poderia devolver o endereço de um dos seguintes nós: 8,17 ou 21. O tipo **ABin** está declarado da seguinte forma.

```
typedef struct nodo {  
    int valor;  
    struct nodo *esq, *dir;  
} *ABin;
```

